



Battleship Game

General Instructions

The exercise is composed of two sections: **Code & Core Implementation** and **Advanced System Design**. Each section is made up of multiple tasks. The candidate is expected to complete all tasks, as they collectively cover the full Advanced Senior Software Engineer (ASSE) assessment matrix.

For the back-end implementation, **JavaScript and/or TypeScript is the mandatory language**. You are not expected or evaluated on data-base or micro-services management; a simple back-end service in Node can be developed. **The business logic code on the backend will be evaluated to the same standard as the frontend code.**

You must do tasks in order. Start with Exercise 1, then 2, and so on. Tasks increase in complexity and scope.

Repository & Submission Guidelines

You are required to create a private repository within the **eDO GitHub workspace** to store all the source code for this assessment. **Once created, you must grant access to this repository to your assigned technical interviewers so they can review your work.** Additionally, all the specific documentation files requested throughout this exercise (e.g., SOLUTION-1 .md, SOLUTION-2 .md, etc.) must be hosted within this repository.

Tech Stack & Third-Party Libraries

While developing your solution, please adhere to the following guidelines regarding third-party tools:

- **Allowed:** You are welcome to use standard web development libraries and frameworks. This includes UI libraries (e.g., React, Vue, Angular), CSS frameworks/libraries (e.g., Tailwind, Sass, Styled Components), and general-purpose backend frameworks (e.g., Express, NestJS).
- **Strictly Forbidden:** The use of specialized game development libraries, game engines, or physics engines (such as Phaser, PixiJS, Three.js, etc.) is not allowed. The core game logic, state management, grid interactions, and scoring systems must be engineered by you from scratch.

Section I: Code & Core Implementation Instructions

This section requires you to define and implement the core logic for the Battleship Game online. The objective of the game is to develop a web application that allows matches between two players in real time. The system should manage the communication between both clients, validate the movements and maintain the state of the game in a synchronous way. You should offer a solution for CSS scoping / class name collision, although comprehensive UI is not the focus. You may use recent Web APIs and are expected to demonstrate strong JavaScript fundamentals and testing practices.

For each task, create a corresponding **SOLUTION-X.md** file documenting design, coding, testing, and file organization decisions that you deem relevant.

Exercise 1: Game Core Logic and Data Structures

I. How the Game Works: Step-by-Step Flow

🎨 See the **appendix** at the end of the document for the mockups.

The game is a classic naval combat simulation adapted for real-time online play. Here is the exact flow from the start of a session to the end of a match:

1. Matchmaking (Lobby Phase) This is strictly an **online game** designed for Player vs. Player (PvP) interactions. There is no offline mode, and you only play against **real players** (there is no need to develop a computer/AI player). Users can join existing games. In this phase, the user can also create new games with the following configuration:

- Game mode: description in Architectural & Design Requirements section. **Simple** is the default option.
- Fleet definition: the number of ships per each ship type. **One of each** is the default definition.
- Turn timer: the maximum number of seconds per turn. During the game if the timer is reached the user loses the turn and the opponent plays. **60 seconds** is the default time.

2. Fleet Placement (The Start) Once two players are matched, the game begins with the setup phase. Each player is presented with an empty 8x8 grid (columns 1-8, rows A-H). Players must position their designated fleet of three ship types:

-  Cruiser (3 cells)
-  Destroyer (2 cells)
-  Submarine (1 cell)

Crucial Rule: Ships can only be placed horizontally or vertically. **They cannot be placed on a diagonal.**

3. The Battle (Gameplay) Once both players lock in their fleets, the shooting phase begins.

- **Visibility:** During the match, the **user sees both boards** on their screen. They see "My Board" (displaying their own ships and the enemy's incoming shots) and the "Enemy Board" (a tracking board where they record their own hits and misses against the opponent's hidden fleet).
- **Turn mechanics:** Players take turns selecting coordinates on the enemy's grid. The system immediately calculates if the shot is a "Hit" (strikes a ship) or a "Miss" (hits water), updating the scoring engine and the UI in real-time.
- **Game events:** for **Hits**, **Misses** and **Ship Sinks** we have to show a visual effect in both boards:
 - If I send a bomb to my opponent I will see the effect in the **Enemy Board** of my screen and the opponent will see the effect in their **My Board**. See appendix for mock-ups. This applies for Hits, Misses and Ship Sinks.

4. The Resolution (Game End) The game ends immediately when one player successfully hits every single cell of the entire enemy fleet, completely sinking the opponent's fleet. A victory screen is displayed, and the final state of both boards is revealed.

II. Architectural & Design Requirements

Beyond the core logic, the development of this application must adhere to the following strategic and technical guidelines:

- **Web App Store Publishing:** The final product is intended to be published in a Web Application Store in the future. The code must be production-ready, secure, and optimized for a public release.
- **Scalability for Multiple Teams:** The architecture must be highly modular. **Multiple teams will work on this web app** simultaneously in the future, so enforcing a clean separation of concerns, minimal coupling, and maximum cohesiveness is mandatory.
- **Mobile-First Approach:** The user interface must be designed **mobile-first**. The layout, interactions (like drag-and-drop), and visual hierarchy must be perfectly

optimized for touchscreens and smaller viewports before scaling up to desktop displays.

- **Dynamic UI Customization:** The system must be built to **accept UI customization** and theming easily. The design should allow for seamless CSS scoping and swapping of styles without touching the core game logic (For example: The marketing team needs to be able to apply a special "Christmas style" skin to the entire game interface during the holiday season).

This exercise focuses on defining the internal representation of the game board, ships, and implementing the scoring engine.

You are expected to:

1. **Split FE and BE Responsibilities:** Clearly define and implement which business logic resides on the Frontend and which resides on the Backend, ensuring a clean separation of concerns.
2. **Define API Interface:** Define and implement the full API interface (e.g., endpoint definitions, request/response payloads) for all core game actions (lobby, ship placement, shooting, etc.).
3. **Define Ships Structure:** Create the data structures to represent the board (default 8x8) and the ships (Submarine: 1 Cell, Destroyer: 2 Cells, Cruiser: 3 cells). The design must allow new ship sizes/forms to be easily added (extensibility).
4. **Define who should start the game:** implement a logic to agree on who should make the first movement, it can be implemented in different ways, like throwing a dice.
5. **Implement Placement and Hit Logic:** Implement the backend logic to place ships (horizontal or vertical) and determine a hit/miss location.
6. **Scoring System:** the candidate must implement a **dynamic scoring algorithm** that rewards precision, speed, and strategic play. The score must be **visible at all times** on the UI and updated in real time after every interaction.
 - **Dynamic accuracy bonus:** the points awarded for a hit shouldn't be static. The algorithm must calculate the score based on the **probability of success**.
 - i. The lower the probability of hitting a ship (calculated by the ratio of **remaining un-hit ship cells** to the **total remaining hidden cells on the board**), the higher the reward.
 - ii. High risk shots that result in a hit should grant significantly more points than safe or predictable shots.
 - iii. Use a Base hit value of 10 points for the calculations.
 - **Multipliers:** implement a multiplier system to reward performance streaks:
 - i. *Consecutive hits*: successive hits without missing must apply an incremental multiplier (e.g. x1.5, x2, x3). The multiplier resets to x1 upon a miss.
 - ii. *Time-to-action*: if a player records a hit within the first **3 seconds** of

- their turn, a "reflex bonus" multiplier should be applied to that shot's score.
- **Penalties:** any shot that hits the water must result in a **point deduction**. The algorithm should ensure the score doesn't drop below zero.
 - **UI/UX requirements:** the total score must be displayed prominently. The interface must reflect score updates and active multipliers immediately after every shot to provide instant feedback to the player.
7. **Game Modes:** the application must support three distinct game modes. The scoring engine should adjust its logic based on the selected mode.
- **Fastest and Precise (elite mode):** this is the full experience. The algorithm must incorporate all advanced scoring rules:
 - i. Dynamic accuracy.
 - ii. Consecutive hits multiplier.
 - iii. Reflex bonus.
 - iv. Water penalties.
 - **Simple (classic mode):** a straightforward implementation for basic testing:
 - i. Standard scoring: 1 point per hit.
 - ii. No bonuses.
 - iii. No penalties.
 - **Penalties (risk mode):** a mode focused on accuracy where every mistake is costly:
 - i. Scoring: 10 points per hit.
 - ii. Water penalties: every miss results in a -1 point deduction.
 - iii. Zero floor: the score should be updated in real-time but cannot drop below 0.

Expected Deliverables:

1. Fully working code for the game core logic and scoring engine.
2. A file **SOLUTION-1.md** documenting:
 - a. The choice of **Data Structures** for the board and ships, justifying the choice in terms of memory and CPU usage for optimal performance.
 - b. How the code maximizes simplicity and readability.
 - c. The complete API specification.
 - d. A justification for the choice of communication protocol (**WebSocket vs. http requests vs. Server Sent Events**) for real-time interaction, including advantages and disadvantages.

Exercise 2: Testing Strategy and Coverage

This exercise requires a robust testing strategy that validates the game's core logic and scoring engine.

You are expected to:

1. **Implement a Comprehensive Test Suite:** Write tests for the core logic implemented in Exercise 1.
2. **Ensure Full Coverage:** Design a testing plan that validates all business cases. Specify in which step of the development cycle each kind of test must be developed and executed.

Expected Deliverables:

1. A working test suite using appropriate testing utilities.
2. A file `SOLUTION-2.md` providing a detailed rationale test plan for the following testing decisions.

Section II: Advanced System Design Instructions

This section requires you to perform architectural and design work. You do not need to write any code, but you must use **Class** and **Sequence diagrams** to illustrate your design. The solutions must be scalable and consider the performance implications for a high-volume system.

For each task, create a corresponding `SOLUTION-X.md` file documenting the design decisions, patterns used, and architectural considerations.

Exercise 3: Manage Concurrency and Disconnections

Design and document the solutions for:

- In the lobby page, how to handle **concurrent requests to join the same game**.
- During the game, how to **handle a user disconnection** gracefully: Structure this into two proposals:
 - i. Fast iteration (*MVP*)
 - ii. Definitive iteration (*Target picture*)

Expected Deliverables:

A file `SOLUTION-3.md` detailing: explanation of the previous challenges.

Exercise 4: Spectator Mode

Design the architecture for a spectator system that allows external users to watch games through two distinct viewing modalities: **Live Session** & **Old Session**.

Live session: Spectators can view both players' boards, but must not have access to the coordinates of un-hit ships. The spectator should only receive updates for public events (e.g., "Shot fired at A5", "Result: Miss" or "Result: Hit").

Old session: Once a game has concluded, all restrictions are lifted. The initial positions of all ships and the full history of shots become public. Spectators must be able to navigate through the history of the match, allowing them to jump to specific turns or replay the sequence of moves from the beginning to the end. You can see the two full boards at each step.

You are expected to:

1. **Design Spectator Mode:** Create a design that allows spectators to connect and watch the game. This design must integrate with the existing game architecture (from Exercise 3) without introducing significant coupling or performance bottlenecks.

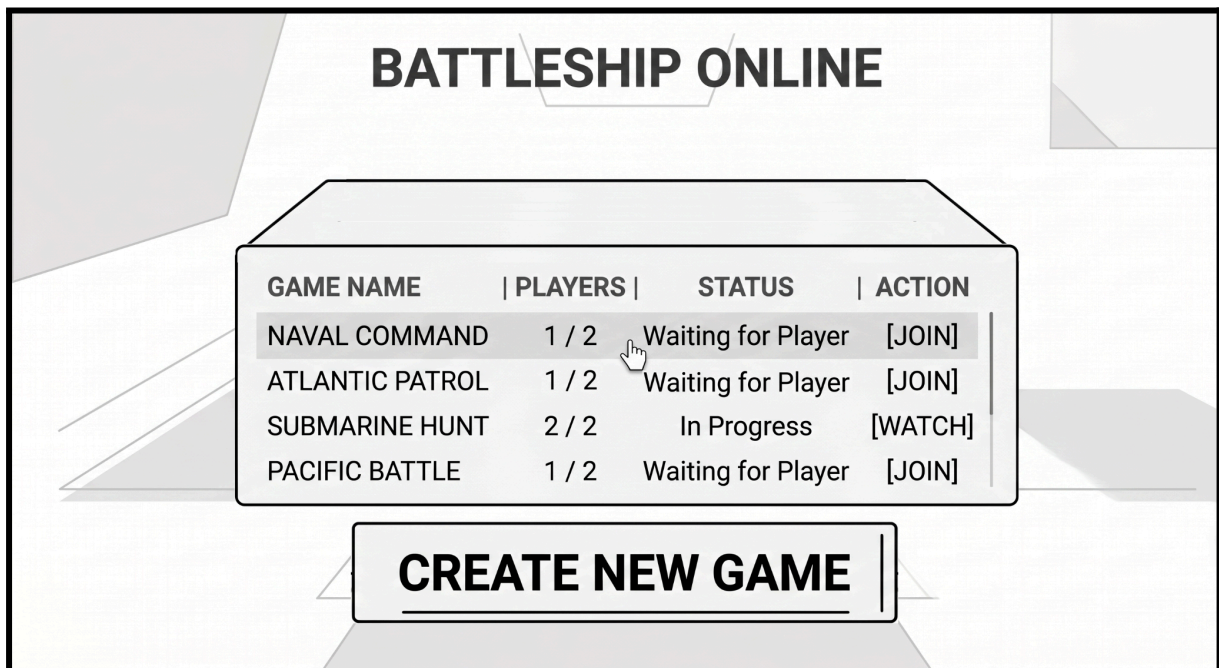
Expected Deliverables:

1. **Class & Sequence Diagrams:** Illustrating the flow from a Player's action to a Spectator's screen.
2. A file **SOLUTION-4.md** detailing:
 - Justification of the chosen **Communication Pattern**, comparing it with at least one alternative.
 - **Monitoring Strategy:** Define specific KPIs for "Player Protection" (ensuring game performance) and "Broadcast Quality" (spectator experience).

Appendix

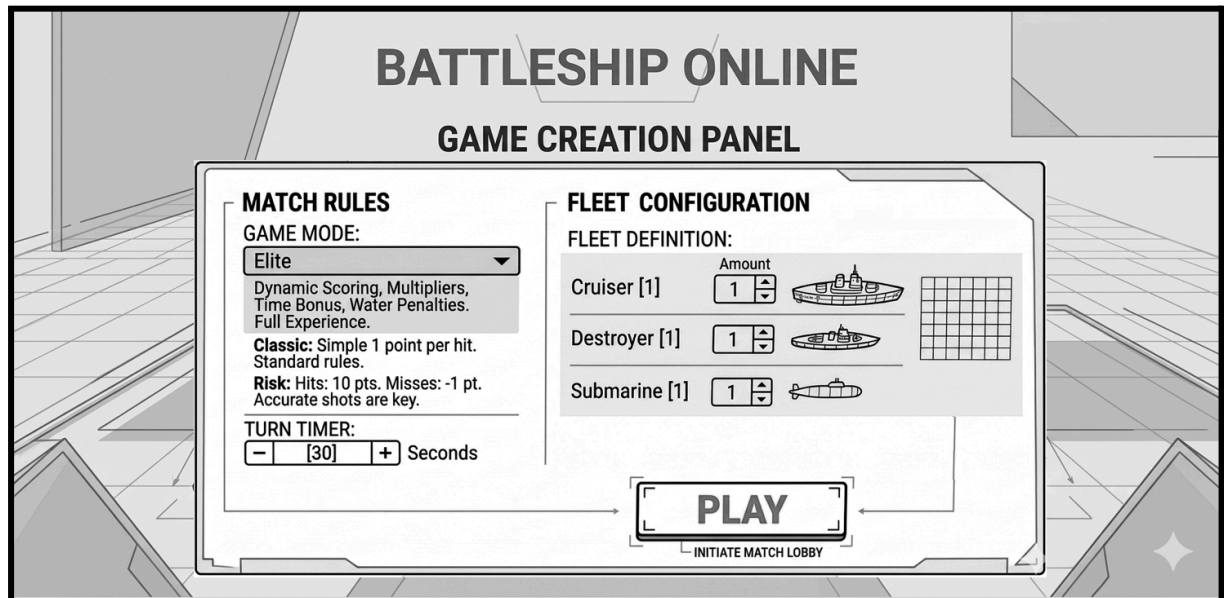
 The provided mockups serve as illustrative guides for game interfaces and usability only; **they do not represent mandatory implementation requirements for the UI.**

Start Screen: The main entry point of the application. It displays a real-time list of available games (showing their status, e.g., "Waiting for player" or "In Progress"). It allows the user to click to join an existing match or click a prominent button to "Create New Game".



 Please note that the Spectator Mode (Exercise 4) is **strictly an architectural design task**. You are not required to implement or write code for WATCH functionality.

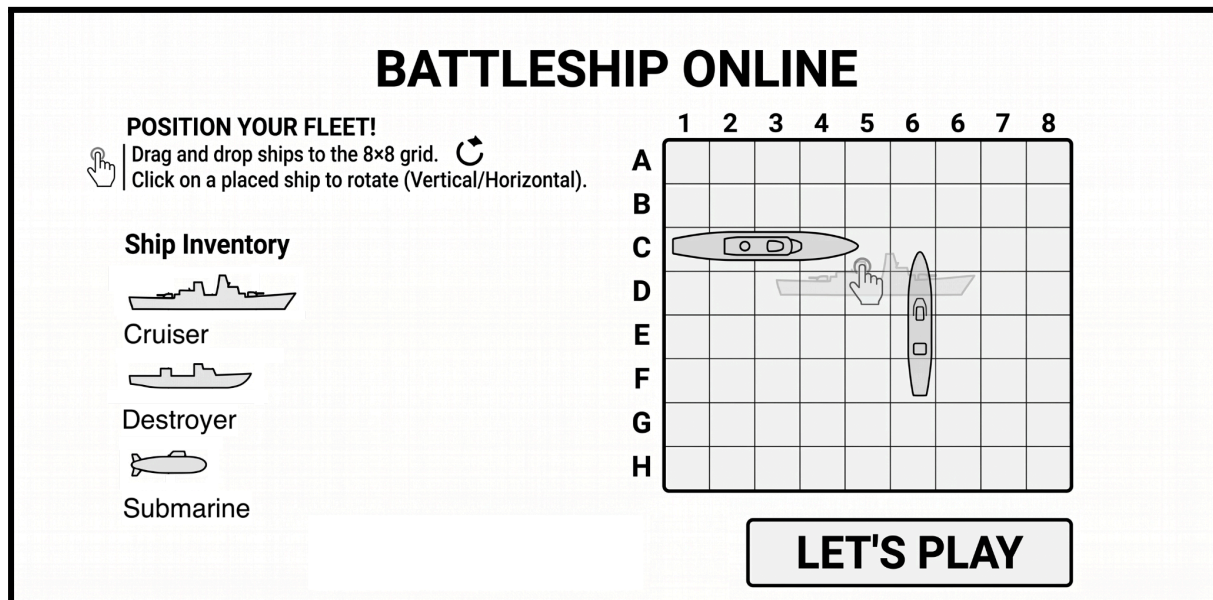
Game Creation Panel: A configuration screen where the user defines the rules for a new match. It includes inputs to select the Game Difficulty/Mode (Elite, Classic, Risk), Fleet Definition (amount of Cruisers, Destroyers, and Submarines), and Turn Timer. It features "Play" (to initiate the lobby).



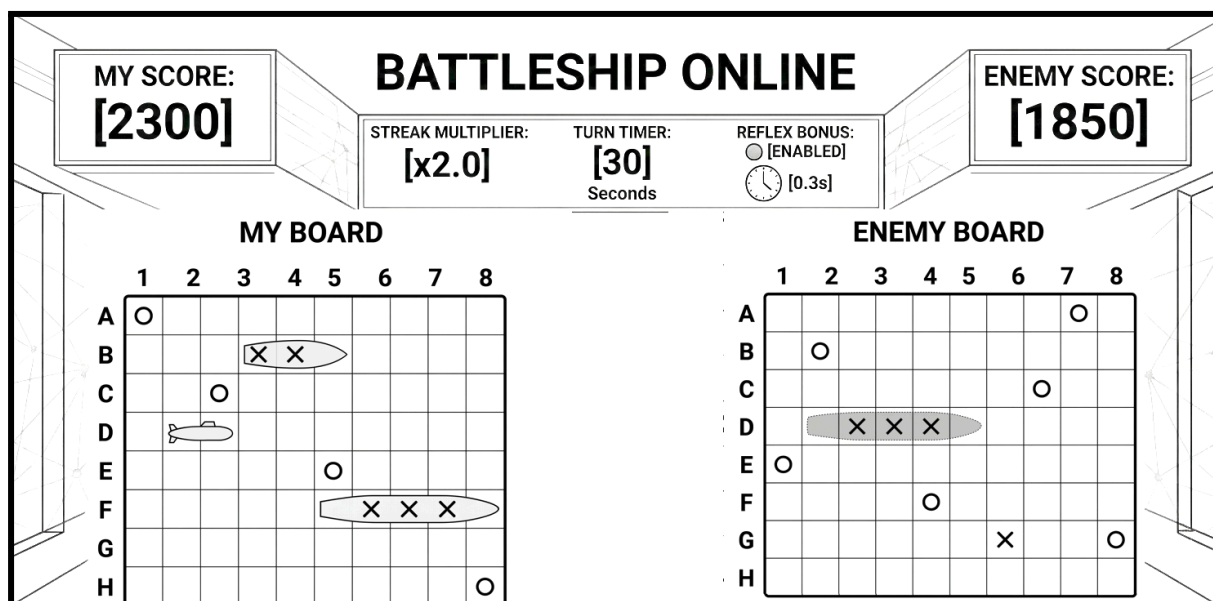
Waiting for Opponent (Lobby): A transitional screen is shown while the system connects the user with a live opponent.



Ship Placement & Setup Phase: The pre-battle interface where players strategically position their fleet on the 8x8 grid. It includes a ship inventory, drag-and-drop functionality, and clear instructions (e.g., click to rotate, no diagonal placements).



Active Gameplay (Battle Phase): The core combat interface. It displays both "My Board" (showing own ships and incoming fire) and the "Enemy Board" (showing only outgoing hits and misses). It must prominently feature the real-time HUD: total scores, turn timer, active streak multipliers, and the "reflex bonus" visual indicator.



Disconnection Handling (*Exercise 3*): A graceful interruption modal that directly addresses the requirements of *Exercise 3*, demonstrating how the system alerts the user and pauses the match when an opponent unexpectedly drops offline.



 Please note that the *Exercise 3* is a design task and *is not supposed to be implemented in code*.

Match Resolution (Victory/Defeat): The final screen declaring the winner. It fully reveals the final state of both boards (showing all hidden ships), displays match statistics, and offers options to "Play Again" or "View Game Replay".



 Please note that the Spectator Mode (Exercise 4) is **strictly an architectural design task**. You are not required to implement or write code for the "Game Replay" feature.